Operating Systems: CPU Scheduling — Fundamentals of CPU Scheduling

1.1 Basic Concepts & Multiprogramming

CPU scheduling is the basis of multiprogramming operating systems. By switching the CPU among processes, the operating system can make the computer more productive. In a uniprocessor system, only one process can run at any time; others must wait until the CPU is free and can be rescheduled.

The success of CPU scheduling depends on the observed property of processes: process execution consists of a cycle of CPU execution and I/O wait. Processes alternate between these two states. Process execution begins with a CPU burst, followed by an I/O burst, followed by another CPU burst, eventually ending on a CPU burst with a system request to terminate.

1.2 Scheduling Criteria

Different CPU scheduling algorithms have different properties, and the choice of an algorithm depends on various criteria. CPU utilization is the percentage of time the CPU is busy, with typical real-world values ranging from 40 percent in lightly loaded systems to 90 percent in heavily loaded environments.

Throughput represents the number of processes completed per unit of time. Turnaround time is the interval from the time of submission of a process to the time of its completion. Waiting time is the sum of periods spent waiting in the ready queue. Response time is the time from the submission of a request until the first response is produced.

Operating Systems: CPU Scheduling — FCFS and SJF Scheduling

2.1 First-Come, First-Served (FCFS) Scheduling

The simplest CPU scheduling algorithm is the First-Come, First-Served (FCFS) algorithm. With this scheme, the process that requests the CPU first is allocated the CPU first. The implementation of the FCFS policy is easily managed with a FIFO queue. When a process enters the ready queue, its PCB is linked onto the tail of the queue.

The average waiting time under the FCFS policy is generally not minimal and may vary substantially if process CPU burst times vary greatly. In particular, the convoy effect occurs when all other processes wait for one big process to get off the CPU, resulting in lower CPU and device utilization than might be possible if shorter processes were allowed to proceed first.

2.2 Shortest-Job-First (SJF) and SRTF

The Shortest-Job-First (SJF) algorithm associates with each process the length of its next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.

SJF is provably optimal because it yields the minimum average waiting time for a given set of processes. The preemptive version of SJF is known as Shortest-Remaining-Time-First (SRTF). In SRTF, if a newly arrived process has a shorter remaining burst time than the currently executing process, the CPU preempts the current process.

Operating Systems: CPU Scheduling — Round Robin and Priority Scheduling

3.1 Round Robin (RR) Algorithm

The Round Robin (RR) scheduling algorithm is designed specifically for time-sharing systems. It is similar to FCFS scheduling, but preemption is added to switch between processes. A small unit of time, called a time quantum or time slice, is defined, generally ranging from 10 to 100 milliseconds.

The ready queue is treated as a circular FIFO queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum. If the process has a CPU burst greater than 1 time quantum, the process is preempted and put back at the tail of the ready queue. The performance of RR depends heavily on the size of the time quantum: if the quantum is extremely large, RR behaves like FCFS; if it is extremely small, context-switch overhead degrades system throughput.

3.2 Priority Scheduling & Starvation

In Priority scheduling, an integer priority is associated with each process, and the CPU is allocated to the process with the highest priority (where lower numbers typically denote higher priority). Equal-priority processes are scheduled in FCFS order.

A major problem with priority scheduling algorithms is indefinite blocking, or starvation. A process that is ready to run can linger indefinitely if higher-priority processes keep arriving. A well-known solution to starvation is aging, a technique of gradually increasing the priority of processes that wait in the system for a long time.

Operating Systems: CPU Scheduling — Multilevel Queues & Multiprocessor Scheduling

4.1 Multilevel Queue and Feedback Queue

A Multilevel Queue (MLQ) scheduling algorithm partitions the ready queue into several separate queues based on process properties like memory size, process priority, or process type (such as interactive foreground versus batch background processes). Each queue has its own scheduling algorithm, such as RR for foreground and FCFS for background.

The Multilevel Feedback Queue (MLFQ) algorithm allows a process to move between queues. If a process uses too much CPU time, it is moved to a lower-priority queue. This scheme leaves I/O-bound and interactive processes in the higher-priority queues. In addition, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue through aging, preventing starvation.

4.2 Multiprocessor Scheduling & Affinity

When multiple CPUs are available, load sharing becomes possible, but the scheduling problem becomes correspondingly more complex. In asymmetric multiprocessing, one master processor handles all scheduling decisions, I/O processing, and other system activities. In symmetric multiprocessing (SMP), each processor is self-scheduling.

Processor affinity refers to a processor's preference to keep a thread running on the same processor to avoid the high cost of invalidating and repopulating cache memory. Soft affinity occurs when an OS attempts to keep a process on a processor without guaranteeing it; hard affinity allows a process to specify a strict subset of processors on which it can run.

Operating Systems: CPU Scheduling — Real-Time Scheduling and Algorithm Evaluation

5.1 Real-Time CPU Scheduling

Soft real-time systems provide no guarantee as to when a critical real-time process will be scheduled; they guarantee only that the process is given preference over noncritical processes. Hard real-time systems have strict deadlines; a task must be serviced by its deadline or the system fails.

In Rate-Monotonic (RM) scheduling, periodic tasks are assigned priorities statically according to their period: the shorter the period, the higher the priority. Earliest-Deadline-First (EDF) scheduling dynamically assigns priorities according to deadlines: the earlier the deadline, the higher the priority. Unlike RM, EDF does not require tasks to be periodic.

5.2 Algorithmic Comparison Summary

Deterministic modeling is an analytic evaluation method that takes a particular predetermined workload and defines the performance of each algorithm for that workload. Simulation involves programming a model of the computer system to measure waiting and turnaround times against synthetic distributions.

Summary of trade-offs: FCFS has zero preemption overhead but suffers from convoy effect. SJF minimizes average waiting time but requires predicting future CPU burst durations. RR provides fair response times but depends critically on quantum selection. MLFQ provides dynamic adaptation for mixed interactive and computational workloads.